



BSc (Hons) in **Information Technology Specialized in AI**

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.

4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent` .
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos` . Call your `astar_search` method and store the returned list of actions in `self.plan` .
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()` . Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS orders its frontier purely by $g(n)$, the cost accumulated so far from the start, and has no idea where the goal actually is. A* orders by $f(n) = g(n) + h(n)$, adding a heuristic estimate of the remaining cost to the goal. That extra $h(n)$ term biases A*'s exploration toward nodes that are actually heading toward the goal, so it expands far fewer irrelevant nodes than UCS, which spreads outward equally in all directions regardless of goal location.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

An admissible heuristic must never overestimate the true remaining cost to the goal. On this grid, movement is restricted to 4 directions, each costing exactly 1 step, and walls can only make the real path longer or equal to the straight grid distance, never shorter. Manhattan distance, $|dx| + |dy|$, is exactly the number of steps needed with no obstacles at all, so it is always a lower bound on the true cost, making it admissible. If the heuristic were not admissible, meaning it sometimes overestimated the true cost, A* could be tricked into skipping over the actual optimal path in favor of a node that only looks cheaper. The algorithm would still terminate and find a path, but it would no longer be guaranteed to find the shortest one.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan distance would no longer be admissible with 8-way movement. With diagonal moves allowed, the agent can cover both an x-step and a y-step in a single move, so the true shortest path can be less than $|dx| + |dy|$. For example, going from (0,0) to (3,3) diagonally takes 3 moves, but Manhattan distance would estimate 6. Since Manhattan would now overestimate the real cost, it is no longer admissible. The correct metric to switch to is Chebyshev distance, $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$, which exactly matches the true cost when diagonal and orthogonal moves cost the same.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Distance to the single nearest food item ignores how much travel is still needed after that pellet is eaten, so it can lead the agent down a path that is locally optimal but globally wasteful, for example picking off a nearby pellet while leaving the rest scattered far apart. A stronger heuristic would estimate the cost to clear the whole remaining food set at once. One option is a Minimum Spanning Tree heuristic, where you build an MST over the agent's current position plus all remaining food positions, using Manhattan distance as edge weights, and use the MST's total weight as $h(n)$. This remains admissible because an MST is a lower bound on the true tour cost needed to visit every food item, and it captures the overall spread of the remaining food rather than just the nearest piece. A cheaper but less tight approximation would be to use the distance to the farthest food item as a lower bound, or nearest food distance plus the minimum distance needed to

connect the rest, which is faster to compute at every node expansion if the grid size makes recomputing a full MST too slow.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING